GhostIQ — Implementation Plan (remaining work)

Version 1 · written 2026-11-02 · repository state: `main` at v0.9.0 + fixes (commit `1089893`)

This document is self-contained: someone (or an AI assistant) with only this file and the repository must be able to understand where GhostIQ stands, how it is built, what is left, and how each remaining item will be implemented and verified. `docs/PLAN.md` holds the original phase plan and research; this file is the working plan from here to v1.0 and beyond.

---

1. What GhostIQ is

GhostIQ is an autonomous, persona-driven QA platform for web apps. Virtual users ("personas": happy path, form stress tester, impatient user, keyboard-only, mobile on slow 3G, unicode, security prober, chaotic explorer) drive a real Chromium browser through an app with an AI planner, while oracles record bugs: runtime errors, failed and slow requests, accessibility (axe-core), security headers and cookies, broken links, redirect loops, duplicate submissions, layout overflow, web vitals, visual regressions. Bugs become deduplicated Findings with evidence. A successful run can be replayed without AI (with locator healing), exported as a plain Playwright test, run in CI with a PR comment, and used by coding agents over MCP.

Source idea: `GhostIQ_Project_Blueprint.pdf` (user's blueprint). GitHub: `https://github.com/Abhishek-dotcom15/Ghost-IQ`.

---

2. Status at a glance

Phase	Name	Status	Tag
0	Foundations (monorepo, CI, Docker)	Done	v0.2.0
1	Provider-agnostic LLM layer	Done	v0.3.0
2	Agent core (aria refs, guardrails, budgets)	Done	v0.4.0
3	Personas as data (YAML, seeded)	Done	v0.5.0
4	Findings engine, oracles, reports, B1 benchmark	Done	v0.6.0
5	Replay, healing, Playwright export, analytics	Done	v0.7.0
6	CLI, GitHub Action, MCP, webhooks, release	Done	v0.8.0
7	Sign-in, roles, API keys, audit, SSRF, retention	Done	v0.9.0
post-7	Verification fixes, plain-language errors, UI redesign	Done	on `main` (untagged)
7b	Postgres, S3 storage, live demo URL	**Left**	v0.10.0
8	Evaluation and launch	**Left**	**v1.0.0**
9	Stretch features	**Left (optional)**	v1.x

Measured so far (demo mode, scripted planner, no AI): B1 seeded-bug benchmark 29/33 bugs (94% of the 31 detectable) at 100% precision with all 8 personas; CI guard (happy path only) 20/31 at 100%. 226 automated

tests. CI green on GitHub; GHCR images public (`ghcr.io/abhishek-dotcom15/ghostiq-{server,runner,mock-app}` tags 0.8.0, 0.9.0, latest). **No real-model numbers exist yet** (no API key has been used).

3. Architecture (as built)

```
apps/server     Express 5 + Socket.IO + Prisma 7 (SQLite, better-sqlite3 adapter) — control plane,
                REST API, findings engine, reports, replay service, analytics, auth, webhooks,
                serves the built dashboard
apps/runner     Runner agent + `ghostiq` CLI: Playwright 1.64 driver, oracles, CLI (run, replay,
                export, issues, mcp, agent, init), MCP server
apps/dashboard  React 19 + Vite 8 dashboard (no router; views switched in App.tsx)
packages/shared zod contracts (socket events, actions, findings, planner, explain.ts)
packages/llm    LLM client: Gemini, Claude, OpenAI-compatible, Ollama, mock; validation + repair,
                retries, fallback chain, metering, record/replay fixtures
packages/core   Agent loop (observe → plan → act → record) over BrowserDriver/Planner
                interfaces; ReplayPlanner + heal ladder
packages/personas  YAML persona loader, 8 built-ins, seeded input generators, behaviours,
                device and network profiles
examples/mock-target-app  demo shop with 33 planted bugs (bugs.json); test hooks
                POST /__reset, POST /__ui {loginLabel}
scripts/        smoke.mjs, bench-bugs.mjs, cli-e2e.mjs, check-dashboard.mjs, auth-e2e.mjs,
                check-links.mjs, check-dockerfile.mjs, lib/stack.mjs
```

Flow: dashboard/CLI → `POST /api/sessions` → server picks the project's connected runner (Socket.IO, API key in handshake) → runner runs `runAgent` with `HttpPlanner` (`POST /api/ai/next-action`, LLM keys stay on the server) → step events stream back → findings engine upserts Findings by fingerprint → run end → run-level oracles, flakiness (replays only), webhook, diagnostics.

Key data (Prisma, `apps/server/prisma/schema.prisma`): Project (public flag), TestSession (mode explore/replay, replayOfId, personaJson, seed, finalStateJson, diagnosticsMd, flakinessScore), TestStep (actionJson, locatorJson, inputJson, healed), NetworkRequest, EndpointHealth, Finding + FindingOccurrence, BaselineImage, HealEvent, LlmCall, Persona, User, Membership, AuthSession, ApiKey, AuditLog.

4. Working conventions and machine facts (read before changing anything)

- **Toolchain:** Node 26 locally (CI/Docker use Node 24), pnpm 12.14 (installed via npm; in PowerShell prepend `$env:APPDATA\npm` and `C:\Program Files\nodejs` to `PATH`), TypeScript ~6.0.3, ESLint 10 flat config, Prettier, Vitest 5, husky + lint-staged on commit.
- **pnpm 12:** `minimumReleaseAge` 14 days (let pnpm choose versions), `allowBuilds` (not `onlyBuiltDependencies`); with `CI=true` installs use a frozen lockfile — locally use `pnpm install --no-frozen-lockfile` after changing dependencies.
- **No Turborepo** (its Windows binary fails); recursive pnpm scripts instead. **No Docker** on the dev machine: Docker builds are verified only in GitHub CI.
- **No bash** on the dev machine (MinGit). **Never edit source files with PowerShell `Get-Content` / `Set-Content`** (Windows PowerShell 5.1 reads UTF-8 as ANSI and corrupts non-ASCII): use the editor tool or Node `fs`. Never name a PowerShell function `R`. `git commit -F -` with a here-string fails: write the message to a file and use `-F file`.
- **Long scratch paths** exceed Windows MAX_PATH for pnpm: clone into `$env:TEMP\gqc`.
- **Prisma:** after schema changes run (in `apps/server`) `node node_modules/prisma/build/index.js

generate `and` db push `. Generated client in` src/generated/prisma`.

- **Ports:** mock app 3000, server 3001, dashboard dev 5173. The e2e scripts start their own stack on 3000/3001: stop any running local instance first.
- **Auth defaults:** `GHOSTIQ_AUTH` defaults to on when `NODE_ENV=production`, off otherwise; the CLI's temporary server and docker compose set it off explicitly. Rate limiting follows auth.
- **Pushing:** each `git push` needs the user's go-ahead. Pushing a `v*.*.*` tag triggers `release.yml` (publishes GHCR images + GitHub release).
- **Definition of done for any change:** `pnpm verify` (format, lint, Dockerfile check, link check, typecheck, tests, build) plus `scripts/smoke.mjs`, `scripts/cli-e2e.mjs`, `scripts/check-dashboard.mjs`, `scripts/auth-e2e.mjs`, and `node scripts/bench-bugs.mjs --personas happy-path --no-visual --steps 80 --min-recall 0.6` all pass; docs updated; honest status in `docs/PLAN.md`.

---

5. Remaining work — overview and order

1. **Phase 8 (required, → v1.0):** evaluation harnesses B2–B5, real-model runs, evaluation write-up, launch material, contributor docs. Needs an LLM API key for the real-model parts; everything else can be built and tested in demo mode first.
2. **Phase 7b (needed for multi-instance/hosted scale):** Postgres, S3-compatible artifacts, live demo deployment. Can run in parallel with Phase 8; the live demo URL is also a Phase 8 launch item.
3. **Phase 9 (optional stretch):** API personas, voice goals, more browser engines, WebMCP testing, shared persona library.

Inputs needed from the user: (a) an LLM API key (`GEMINI_API_KEY` recommended: cheapest; also `ANTHROPIC_API_KEY` for B4), (b) a Render (or other) account for the live demo, (c) optionally a second GitHub repo for the "demo PR" and an OAuth app for GitHub sign-in on the demo.

---

6. Phase 8 — Evaluation and launch (target v1.0.0)

Goal: replace "it works in demo mode" with measured, reproducible numbers on real models and real-ish apps, and make the project easy to adopt and contribute to.

8.1 Benchmark framework (shared)

- New `evals/` layout (outputs stay git-ignored under `evals/results/`): one script per benchmark in `scripts/bench-*.mjs`, all built on `scripts/lib/stack.mjs`.
- Common result format `evals/results/<bench>-<date>-<model>.json`: `{ bench, model, provider,

gitSha, startedAt, params, runs: [...], summary: {...}, costUsd, tokens } `, plus a Markdown table printed to stdout.` scripts/eval-report.mjs `renders all result files into the tables in` docs/EVALUATION.md` (between marker comments) so numbers are never typed by hand.

- Every benchmark supports `--live` (configured model) and demo mode (pipeline check), `--runs`, `--seed`, `--model provider:model`, `--max-cost-usd` (stop when exceeded; uses LlmCall cost).
- Cost guard: refuse to start a live benchmark without `--max-cost-usd`.

8.2 B1 — seeded bugs (exists)

`scripts/bench-bugs.mjs` already measures recall/precision against `examples/mock-target-app/bugs.json`. Remaining: run with `--live` on the default model, record results, add `--json` output in the common format.

8.3 B2 — goal completion on three apps

- Apps: (1) the bundled demo shop; (2) a second bundled app, `examples/todo-app` (Express + static HTML: sign-up, login, create/edit/delete todos, filters, settings page, deliberately realistic labels — no planted bugs); (3) a public, stable practice site where automated testing is allowed (candidates: `https://demo.playwright.dev/todomvc`, `https://the-internet.herokuapp.com`; confirm terms before use; never test sites without permission).
- Task suite `evals/tasks/*.yaml`: per app 8–10 goals with a machine-checkable success condition (`final.url` matches, heading visible, API call observed, or text present), e.g. "Sign up, log in and add two todos" → API `POST /api/todos` twice and both texts visible.
- Metrics per model: success rate (goal met AND success condition true — the agent's own "goal met" is not trusted), median actions, median wall time, cost per run and per success, false "goal met" rate (agent claimed success, condition false).
- Implementation: `scripts/bench-goals.mjs` creates a project per app, runs each task N times (default 3) with the happy-path persona, evaluates the success condition from the stored run (`finalStateJson`, steps, network logs).

8.4 B3 — healing under UI mutations

- Mutation hooks in the demo shop (extend `POST /__ui`): rename button text (similar: "Sign In" → "Sign in now"; dissimilar: → "Continue"), change element type (button → link), reorder form fields, wrap in extra divs, change ids/classes only, remove the element (must fail).
- `scripts/bench-healing.mjs`: record a set of passing runs, then for each mutation replay them and classify: passed without heal (robust locator), healed correctly (heal + end state matches), healed wrongly (heal accepted but end state differs or wrong element), correctly refused (element removed → replay_diverged). Report heal precision/recall and false-heal rate, per ladder rung (fuzzy vs model). Live mode exercises the model rung.

8.5 B4 — model ablation

- `scripts/bench-models.mjs` runs B1 (subset) and B2 across models: Gemini flash-lite (default), a stronger Gemini, Claude Haiku, Claude Sonnet, a local Ollama model (e.g. llama3.2 or qwen), and the mock baseline. Same seeds and tasks.
- Report: success rate, bugs found, precision, median latency per step, cost per run, cost per finding; include a Pareto chart (cost vs success) in the evaluation doc (static SVG generated by `eval-report.mjs`, following the dataviz rules already used in the dashboard).

8.6 B5 — reproducibility and flakiness

- Determinism: same persona + seed ⇒ identical generated inputs and behaviour schedule (already in smoke; extend to all 8 personas). Replay determinism: replay each recorded run 10× ⇒ flakiness should be 0% on the stable demo app; then enable a flaky mode in the demo shop (`/__ui {flaky: 0.3}` randomly delays/fails one endpoint) and check the flakiness score detects it (expected ≈ 2·p·(1−p) scaled).
- Explore-mode variance with a real model: same goal 10× ⇒ distribution of actions and findings (documents how non-deterministic exploration is; replay is the deterministic path).

8.7 Evaluation write-up

`docs/EVALUATION.md` rewritten around B1–B5: method, apps, tasks, models, tables generated by `eval-report.mjs`, honest limitations (demo app is ours; practice sites are small; numbers are per model version and date). README "Results" section quotes the headline numbers with links.

8.8 Launch material

- Live demo: deploy via `render.yaml` (server + runner worker + demo shop), make the demo project public read-only, seed it with runs from 3–4 personas. README links it.
- Demo video (2–3 min) and GIFs: scripted with Playwright's `page.screencast`/video recording driving the dashboard (`scripts/record-demo.mjs`): launch a run, watch it live, open findings, replay with a heal, export a test, PR comment. GIFs via ffmpeg (if available) or keep MP4/WebM.
- A demo PR on a separate repository using the Action (`uses: Abhishek-dotcom15/Ghost-IQ@v1.0.0`) so the README can link a real GhostIQ comment.
- README overhaul: one-paragraph pitch, screenshot/GIF, quick start (Docker, local, CLI, Action), results table, architecture diagram, docs index.

8.9 Contributor experience

`CONTRIBUTING.md` (setup on Windows/macOS/Linux, the verification suite, coding conventions, how to add an oracle / persona / LLM provider), `CODE_OF_CONDUCT.md`, `.github/ISSUE_TEMPLATE/` (bug, feature, persona proposal), `.github/pull_request_template.md`, `SECURITY.md` link, labels, a "good first issue" list (from Phase 9 items), `CHANGELOG.md` generated from tags.

8.10 Release v1.0.0

Checklist: all benchmarks run live and recorded; EVALUATION and README updated; live demo up; docs reviewed; version strings bumped (`apps/runner/src/version.ts`, package versions); `pnpm verify` + all e2e pass; tag `v1.0.0` (publishes images + release notes).

Done when: README shows a live demo link, CI badge and measured real-model numbers; v1.0.0 release published.

---

7. Phase 7b — Scale-out storage and deployment

7b.1 Postgres

- Problem: Prisma generates a provider-specific client; the schema says `sqlite`.
- Plan: keep one schema source; generate a Postgres variant at build time (`scripts/prisma-variant.mjs` writes `prisma/schema.postgres.prisma` with `provider = "postgresql"` and a second generator output `src/generated/prisma-pg`). `db.ts` picks the client by `DATABASE_URL` scheme (`file:` → SQLite + better-sqlite3 adapter; `postgres://` → `@prisma/adapter-pg`). Types are identical across both clients; export one `PrismaClient` type.
- Switch from `db push` to versioned migrations (`prisma migrate`) per provider, with a baseline migration for existing SQLite databases.
- Verification: CI job with a `postgres:17` service container running the server test suite and `smoke.mjs` against Postgres (cannot be tested on the dev machine).
- Watch: SQLite-specific SQL (none known), `groupBy`/`distinct` behaviour, string case sensitivity (Postgres is case-sensitive in `contains`), JSON stored as strings stays portable.

7b.2 S3-compatible artifact storage

- `ArtifactStore` interface (`put`, `get` stream, `delete`, `list(prefix)`, `url`) with `local` (current folders) and `s3` (`@aws-sdk/client-s3`; works with AWS S3, Cloudflare R2, MinIO) implementations, chosen by `GHOSTIQ_STORAGE=s3` + bucket/endpoint/credentials env.

- Screenshots, traces, diff and baseline images go through the store; `/screenshots` and `/artifacts` routes stream from it after the existing access checks (or redirect to short-lived presigned URLs). Retention deletes through the store.
- Verification: unit tests with an in-memory store; CI job with a MinIO service container.

7b.3 Live demo deployment

Deploy `render.yaml` on the user's Render account (paid plan for the disk, or Postgres + S3 once 7b.1/7b.2 land). Configure `GHOSTIQ_PUBLIC_URL`, admin token, optional GitHub OAuth, public demo project. Document in `docs/DEPLOY.md` (already written).

Done when: server tests and smoke pass on Postgres and with S3 storage in CI; the demo URL is live.

---

8. Phase 9 — Stretch (optional, in suggested order)

1. **API personas** (`packages/api-personas`): import OpenAPI 3 or HAR, generate request sequences per persona (valid, boundary, injection, auth-less), run without a browser, reuse the findings engine (new issue kinds: `schema_violation`, `auth_bypass`, `server_error`). CLI: `ghostiq api -s openapi.yaml`.
2. **Browser matrix:** `--browser chromium|firefox|webkit` in the runner and personas (`browser:` field); axe and most oracles work across engines; CDP-only features (network throttling) fall back to route-level delays on Firefox/WebKit.
3. **Voice goals (Whisper):** dashboard microphone button → audio to the server → transcription (OpenAI-compatible `/audio/transcriptions` or local whisper.cpp) → goal text. Optional, behind a setting; never store audio.
4. **Shared persona library:** publish/import personas as signed YAML bundles (GitHub repo or registry URL); version pinning; persona marketplace page in the dashboard.
5. **WebMCP tool testing:** discover tools a site exposes to AI agents and fuzz them with personas (contract, auth and injection checks).
6. Other backlog: vision-LLM judge for layout bugs (deferred since Phase 4; needs triage data to measure precision), diff-aware runs (map changed files to routes), npm package of the CLI (bundle workspace packages with tsup/esbuild), stored alternate locators as the first heal rung, disambiguating repeated identical controls in replay, persistent replay queue, video receipts per finding.

---

9. Risks and mitigations

Risk	Mitigation
Real-model results disappoint	Report honestly with per-task breakdown; improve prompts/snapshot trimming; keep demo-mode numbers separate
LLM cost overruns during benchmarks	Mandatory `--max-cost-usd`, metering already exists, start with flash-lite
Practice sites change or forbid automation	Prefer bundled apps; pin and document external targets; skip if unavailable
Postgres/S3 untestable locally	CI service containers; keep SQLite/local as the default path
Windows-specific tooling issues	Follow section 4; verify on Linux CI
Model IDs retired	Model IDs come from env; boot health check prints the fix

10. Quick restart guide (if context is lost)

1. Read this file, then `docs/PLAN.md`, `docs/ARCHITECTURE.md`, `docs/SECURITY.md`, `docs/CI.md`, `README.md`.
2. `git log --oneline` and `git tag` to see the latest state; compare with section 2.
3. Set up: `pnpm install`, `pnpm db:setup`, `pnpm --filter @ghostiq/runner browsers:install`, `pnpm build`.
4. Run the verification suite (section 4) to confirm a green baseline.
5. Run locally: mock app (`node examples/mock-target-app/server.js`), server (`GHOSTIQ_DASHBOARD_DIR=apps/dashboard/dist node apps/server/dist/index.js` from `apps/server`), runner (`node apps/runner/dist/index.js -k local-app-test-project-id -u http://localhost:3001`); dashboard at http://localhost:3001.
6. Continue with the first unchecked item in section 11.

11. Task checklist

Phase 8

- ☐ 8.1 Common result format + `--json` + `eval-report.mjs` + cost guard
- ☐ 8.2 B1 live run recorded
- ☐ 8.3 `examples/todo-app`, task suite YAML, `bench-goals.mjs`, live run recorded
- ☐ 8.4 Mutation hooks, `bench-healing.mjs`, live run recorded
- ☐ 8.5 `bench-models.mjs`, Pareto chart, runs across 4–6 models
- ☐ 8.6 Reproducibility across personas, flaky mode, explore-variance study
- ☐ 8.7 `docs/EVALUATION.md` rewritten from generated tables
- ☐ 8.8 Live demo, `record-demo.mjs` video/GIFs, demo PR, README overhaul
- ☐ 8.9 CONTRIBUTING, CoC, issue/PR templates, CHANGELOG
- ☐ 8.10 v1.0.0 released

Phase 7b

- ☐ 7b.1 Postgres variant client, migrations, CI Postgres job
- ☐ 7b.2 ArtifactStore (local + S3), CI MinIO job
- ☐ 7b.3 Live deployment on Render

Phase 9

- ☐ API personas · [ ] browser matrix · [ ] voice goals · [ ] persona library · [ ] WebMCP · [ ] backlog items